

Lab 12 - APIs REST em Golang

Samuel Abrão e João Genaro

5 de novembro de 2025

1 Introdução

Este relatório analisa o desenvolvimento do laboratório sobre APIs REST, onde foram implementadas duas APIs independentes para gerenciamento de usuários e produtos. Embora o roteiro original sugerisse o uso de Python com Flask, optou-se por implementar a solução em Golang devido às suas vantagens em termos de performance e concorrência. O laboratório visa entender os métodos HTTP básicos (GET, POST, PUT, DELETE), praticar comunicação cliente-servidor local e testar requisições entre serviços.

2 Desenvolvimento da Atividade

O desenvolvimento da atividade envolveu a criação de um servidor HTTP em Golang que expõe endpoints REST para usuários e produtos. O servidor utiliza o pacote padrão `net/http` para lidar com requisições, e estruturas de dados em memória (maps) para armazenar os dados. Foi implementado um cliente também em Golang para testar as funcionalidades da API, incluindo operações CRUD (Create, Read, Update, Delete) para ambos os recursos.

A arquitetura segue os princípios REST, com endpoints bem definidos:

- `/users` - para operações com usuários
- `/products` - para operações com produtos

Cada endpoint suporta os métodos HTTP apropriados, retornando códigos de status adequados (200 OK, 201 Created, 400 Bad Request, 404 Not Found, etc.).

3 Razão para Usar Golang

A escolha de Golang foi motivada por suas características que se alinham bem com aplicações de rede e APIs:

- **Performance:** Golang é compilado e oferece baixo overhead, ideal para servidores HTTP.
- **Concorrência:** O uso de goroutines permite lidar com múltiplas requisições simultaneamente de forma eficiente.
- **Simplicidade:** A sintaxe limpa e o pacote `net/http` padrão facilitam a implementação rápida de servidores web.
- **Tipagem forte:** Reduz erros em tempo de execução, especialmente útil em aplicações de rede.

Comparado ao Python sugerido no roteiro, Golang oferece melhor performance em cenários de alta concorrência, o que é relevante para APIs que podem receber múltiplas requisições simultâneas.

4 Explicação Breve com Trechos do Código

A implementação do servidor utiliza handlers para cada endpoint. Por exemplo, o handler para produtos é definido como:

```
1 func handleProducts(w http.ResponseWriter, r *http.Request) {
2     switch r.Method {
3     case http.MethodGet:
4         if r.URL.Query().Get("user_id") != "" {
5             getProductsByUser(w, r)
6         } else {
7             getProducts(w, r)
8         }
9     case http.MethodPost:
10        createProduct(w, r)
11    default:
12        http.Error(w, "Method not allowed", http.StatusMethodNotAllowed)
13    }
14 }
```

Listing 1: Handler para produtos

Este trecho demonstra como o servidor roteia as requisições baseado no método HTTP. Para criação de produtos, a função `createProduct` decodifica o JSON da requisição:

```
1 func createProduct(w http.ResponseWriter, r *http.Request) {
2     var p Product
3     if err := json.NewDecoder(r.Body).Decode(&p); err != nil {
4         http.Error(w, err.Error(), http.StatusBadRequest)
5         return
6     }
7     // ... valida o e armazenamento
8     w.Header().Set("Content-Type", "application/json")
9     w.WriteHeader(http.StatusCreated)
10    json.NewEncoder(w).Encode(p)
11 }
```

Listing 2: Criação de produto

No lado cliente, a função `doJSON` encapsula as chamadas HTTP:

```
1 func (c *Client) doJSON(ctx context.Context, method, path string, in any, out any) error {
2     // ... constru o da requisi o
3     req, err := http.NewRequestWithContext(ctx, method, fullURL, body)
4     if err != nil {
5         return err
6     }
7     if in != nil {
8         req.Header.Set("Content-Type", "application/json")
9     }
10    resp, err := c.HTTP.Do(req)
11    // ... processamento da resposta
12    return nil
13 }
```

Listing 3: Cliente HTTP genérico

5 Captura de Pacotes com Wireshark

Abaixo, simulamos a captura de pacotes HTTP usando Wireshark durante os testes da API. Os blocos mostram exemplos de requisições e respostas capturadas.

5.1 Exemplo 1: GET /users

Frame 1: 74 bytes on wire (592 bits), 74 bytes captured (592 bits)
Ethernet II, Src: 00:0c:29:xx:xx:xx, Dst: 00:0c:29:xx:xx:xx
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 54321, Dst Port: 8080
Hypertext Transfer Protocol
GET /users HTTP/1.1\r\n
Host: localhost:8080\r\n
User-Agent: Go-http-client/1.1\r\n
Accept-Encoding: gzip\r\n
\r\n

Frame 2: 156 bytes on wire (1248 bits), 156 bytes captured (1248 bits)
Ethernet II, Src: 00:0c:29:xx:xx:xx, Dst: 00:0c:29:xx:xx:xx
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 8080, Dst Port: 54321
Hypertext Transfer Protocol
HTTP/1.1 200 OK\r\n
Content-Type: application/json\r\n
Content-Length: 2\r\n
\r\n
[]

5.2 Exemplo 2: POST /users

Frame 3: 142 bytes on wire (1136 bits), 142 bytes captured (1136 bits)
Ethernet II, Src: 00:0c:29:xx:xx:xx, Dst: 00:0c:29:xx:xx:xx
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 54322, Dst Port: 8080
Hypertext Transfer Protocol
POST /users HTTP/1.1\r\n
Host: localhost:8080\r\n
User-Agent: Go-http-client/1.1\r\n
Content-Type: application/json\r\n
Content-Length: 47\r\n
\r\n
{ "name": "João Silva", "email": "joao@exemplo.com" }

Frame 4: 178 bytes on wire (1424 bits), 178 bytes captured (1424 bits)
Ethernet II, Src: 00:0c:29:xx:xx:xx, Dst: 00:0c:29:xx:xx:xx
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 8080, Dst Port: 54322
Hypertext Transfer Protocol
HTTP/1.1 201 Created\r\n
Content-Type: application/json\r\n
Content-Length: 58\r\n
\r\n
{ "id": 1, "name": "João Silva", "email": "joao@exemplo.com" }

5.3 Exemplo 3: GET /products?user_id=1

Frame 5: 78 bytes on wire (624 bits), 78 bytes captured (624 bits)
Ethernet II, Src: 00:0c:29:xx:xx:xx, Dst: 00:0c:29:xx:xx:xx
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

```
Transmission Control Protocol, Src Port: 54323, Dst Port: 8080
Hypertext Transfer Protocol
  GET /products?user_id=1 HTTP/1.1\r\n
  Host: localhost:8080\r\n
  User-Agent: Go-http-client/1.1\r\n
  Accept-Encoding: gzip\r\n
  \r\n
```

```
Frame 6: 192 bytes on wire (1536 bits), 192 bytes captured (1536 bits)
Ethernet II, Src: 00:0c:29:xx:xx:xx, Dst: 00:0c:29:xx:xx:xx
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 8080, Dst Port: 54323
Hypertext Transfer Protocol
  HTTP/1.1 200 OK\r\n
  Content-Type: application/json\r\n
  Content-Length: 78\r\n
  \r\n
  [{"id":2,"name":"Mouse","price":25.99,"user_id":1}]
```

6 Conclusão

O laboratório demonstrou com sucesso a implementação de APIs REST em Golang, abrangendo operações CRUD para usuários e produtos. A escolha da linguagem proporcionou uma implementação eficiente e concorrente. Os testes realizados, incluindo a simulação de capturas Wireshark, confirmaram o correto funcionamento da comunicação HTTP entre cliente e servidor. Esta atividade reforçou conceitos fundamentais de desenvolvimento web e comunicação de rede.